



Universidade Federal de Uberlândia

## - 1º Atividade avaliativa -

*Professores Alan Petrônio, Marcelo Rodrigues, Kil Jin, Renato Carrijo*

Faculdade de Engenharia Elétrica

Versão 1.0 - 2026/1

### 1) Apresentação do problema a resolver:

Deve ser desenvolvido um programa de computador, em linguagem Python, que permita realizar operações de cadastro, consulta, atualização e remoção (CRUD) de ativos de TI e de vulnerabilidades associadas a esses ativos. A aplicação deverá simular, de forma simplificada, uma base de inventário de segurança utilizada por uma equipe de Computação ou Cibersegurança para acompanhar quais equipamentos, sistemas ou serviços existem em uma organização, quais vulnerabilidades foram identificadas neles e qual é a situação de tratamento dessas vulnerabilidades.

#### ***Sobre CRUD***

##### **Letra (Termo em inglês)**

C (Create)

R (Read)

U (Update)

D (Delete)

##### **Tradução usual**

Criar/Cadastrar

Ler/Consultar

Atualizar

Excluir/Remover

##### **Exemplo no trabalho**

Cadastrar um novo ativo de TI.

Buscar e visualizar os dados de um ativo.

Alterar responsável, localização ou status de um ativo.

Remover um ativo da base de dados.

Neste trabalho, considera-se como ativo de TI qualquer recurso computacional que precise ser acompanhado pela organização, como notebooks, servidores, estações de trabalho, roteadores, impressoras de rede, sistemas internos, softwares licenciados, aplicações web ou bancos de dados. Cada ativo deverá possuir um identificador único no sistema, além de informações básicas que permitam sua localização, classificação e consulta.

Também deverá ser possível associar vulnerabilidades aos ativos cadastrados. Uma vulnerabilidade representa uma fragilidade ou problema que pode comprometer a segurança de um ativo, como falha de configuração, ausência de atualização, senha fraca, exposição indevida de serviço, software desatualizado ou permissão de acesso inadequada. O estudante poderá consultar fontes confiáveis para conhecer outros exemplos e ampliar sua visão sobre o assunto. Para fins deste trabalho, as vulnerabilidades não precisam ser reais nem obtidas de bases externas; elas podem ser cadastradas manualmente pelo usuário, desde que contenham informações suficientes para permitir sua classificação, consulta e acompanhamento.

O programa deve atender aos requisitos listados na Tabela 1, a seguir.

Tabela 1 - Lista de requisitos para a aplicação requerida

| Nº | Requerimento | Peso (%) |
|----|--------------|----------|
|----|--------------|----------|

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |     |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 1  | O programa deve possuir um prompt, menu textual ou interface gráfica para interagir com o usuário. No caso de prompt ou menu textual, devem ser apresentadas opções claras para as operações disponíveis. As entradas de dados devem possuir tratamento de erros para evitar falhas causadas por comandos inválidos, campos vazios ou tipos de dados incorretos.                                                                                                                          | 10  |
| 2  | Criar no programa uma lista de tipos de ativos de TI com pelo menos 4 categorias, utilizando uma estrutura de dados de enumeração. Exemplos de categorias possíveis são: notebook, servidor, roteador, software licenciado, aplicação web, banco de dados, impressora de rede ou estação de trabalho. A cada tipo de ativo deve estar associado um código inteiro para referência no sistema.                                                                                             | 10  |
| 3  | Ler do prompt um comando de cadastro de ativo de TI contendo, no mínimo, identificador único do ativo, nome ou hostname, responsável, setor ou localização, tipo de ativo e lista inicial de vulnerabilidades associadas, quando houver. Essas informações devem ser gravadas ou concatenadas em um ou mais arquivos de texto, que funcionarão como base de dados da aplicação. O identificador do ativo deve ser um número inteiro e deve ser usado como identificador único no sistema. | 10  |
| 4  | Buscar na base de dados os registros de um ativo solicitado pelo usuário e exibi-los de forma organizada. A busca deve poder ser realizada, no mínimo, pelo identificador do ativo ou pelo nome/hostname.                                                                                                                                                                                                                                                                                 | 10  |
| 5  | Atualizar os dados de um ativo de TI já cadastrado na base de dados, com base em novas informações inseridas pelo usuário no prompt. A atualização poderá envolver, por exemplo, mudança de responsável, setor, localização, tipo de ativo ou descrição do ativo.                                                                                                                                                                                                                         | 10  |
| 6  | Deletar o registro de um ativo de TI da base de dados, removendo também as vulnerabilidades associadas a esse ativo, quando existirem.                                                                                                                                                                                                                                                                                                                                                    | 10  |
| 7  | Permitir cadastrar, a qualquer momento após o cadastro de um ativo, vulnerabilidades associadas a ele. Para cada vulnerabilidade, o programa deverá registrar pelo menos uma descrição, uma categoria ou tipo, uma severidade e um status de tratamento. A severidade pode seguir uma escala simples, como baixa, média, alta e crítica. O status pode indicar, por exemplo, se a vulnerabilidade está aberta, em tratamento, corrigida ou aceita como risco..                            | 10  |
| 8  | Visualizar as vulnerabilidades associadas a um ativo de TI, exibindo, quando houver vulnerabilidades cadastradas, sua descrição, severidade e status de tratamento. Quando não houver vulnerabilidades associadas ao ativo, o sistema deve informar que o ativo está sem vulnerabilidades registradas                                                                                                                                                                                     | 5   |
| 9  | Utilizar uma estrutura de dicionário (em Python, por exemplo, temos o tipo dict, uma estrutura do tipo hash map) para otimizar buscas, consultas ou organização interna dos registros. Por exemplo, o dicionário pode indexar ativos pelo identificador único, pelo hostname ou por outro campo considerado relevante.                                                                                                                                                                    | 10  |
| 10 | O código deve estar disponível no GitHub, Bitbucket ou plataforma similar. No repositório, o estudante deve demonstrar que, durante a produção do trabalho, soube trabalhar com mais de 2 branches e realizar merge entre eles.                                                                                                                                                                                                                                                           | 15  |
| 11 | Optativo: Utilizar um banco de dados relacional no projeto (aumenta o valor da nota).                                                                                                                                                                                                                                                                                                                                                                                                     | 10* |

\*pontos na final, conforme Equação II no final deste documento.

## 2) Sobre a natureza do projeto:

- Objetivo: verificar de maneira prática se você conseguiu aprender a programar e usar conceitos de programação corretamente para resolver um problema e como você estruturou sua ideia.
- O que se espera do seu projeto:
  1. que ele seja interativo com o usuário e funcional, ainda que a interface gráfica seja limitada
  2. que ele não apresente erros e faça tratamento de erros e exceções
  3. que ele use as estruturas de dados corretas

### 3) Condições:

- A. Os trabalhos são individuais.
- B. Códigos ou arquiteturas de soluções iguais entre estudantes terão notas zeradas.
- C. O programa deve ser implementado em Python
- D. Não é proibido o uso moderado de ferramentas de IA para gerar código parte da solução. Todavia, lembre-se que o que está sendo avaliado não é o código, mas sim como você organizou as ideias, usou os recursos que aprendeu e, principalmente, sabe explicar como usou isto e porque usou, demonstrando consciência de uso e não só mera injeção de código.
- E. Esta avaliação equivale aos ciclos (scrum) 1 e 2 de aprendizagem.

### 4) Formato apresentação e data de entrega:

- A. Os códigos devem estar disponíveis na sua conta do GitHub/bitbucket até o dia **1/junho/26** as 19h. Qualquer outro código em data posterior será desconsiderado!
- B. Será necessário apresentar somente o código fonte no repositório (aberto ao docente) e ele sendo executado no seu computador durante a apresentação. Nenhum outro documento é necessário.
- C. O seu tutor/avaliador reunirá individualmente com cada um dos seus estudantes e concederá a cada um 5 minutos para apresentar o programa em execução, demonstrando o que funciona e o que não funciona, com base nos requisitos. Posteriormente, o docente fará a análise do seu código fonte e da forma como foi estruturada a sua solução, podendo fazer perguntas gerais sobre o trabalho realizado.

### 5) Quantificação da avaliação:

A avaliação final será feita com base nos critérios da Tabela 2.

Tabela 2 – Quadro de avaliação das atividades do projeto

|              | Peso nota final | Atividade                                                                               | Composição item | Excelente (100%) | Bom (75%) | Regular (50%) | Ruim (25%) | Péssimo (0%) | Não aplicável |
|--------------|-----------------|-----------------------------------------------------------------------------------------|-----------------|------------------|-----------|---------------|------------|--------------|---------------|
| Apresentação | 30%             | <b>A1</b> – Apresentação consistente, demonstrando segurança e conhecimento do que fez. | 100%            |                  |           |               |            |              |               |
|              | 70%             | <b>B1</b> - Atendimento aos requisitos (número de requisitos atendidos a contento)      | 60%             | Não aplicável    |           |               |            |              |               |

| Atendimento aos | B2 - Elegância técnica da solução | 20% |  |  |  |  |  |  |
|-----------------|-----------------------------------|-----|--|--|--|--|--|--|
|                 | B3 - Capricho do trabalho         | 20% |  |  |  |  |  |  |

A nota final ( $Nota_{final}$ ) do estudante será dada por:

$$Nota_{trab} = 30. \left( \frac{A1}{100} \right) + \frac{70}{100} \cdot \left( 60. \frac{B1}{100} + 20. \frac{B2}{100} + 20. \frac{B3}{100} \right) \quad (I)$$

$$Nota_{final} = 0,9. Nota_{trab} + 0,1. N_{trabDiarios} + O_{pt} \quad (II)$$

onde:

- $O_{pt}$  são os pontos obtidos de maneira optativa. Neste caso,  $O_{pt}=10$  pontos para alunos que atenderem ao requisito 11.
- $B1 = \sum \text{PesoRequisitos}$  (tabela 1)
- $N_{traDiarios}$  é a avaliação dos trabalhos diários desenvolvidos pelo discente nas *sprints*, de valor 0 a 100 pontos, feita pelo respectivo tutor/professor.

### Exemplo

Para ilustrar, considere um estudante hipotético que tenha cumprido satisfatoriamente os oito primeiros requisitos. Nesse caso, o seu peso acumulado ( $B1$ ) é 75. Supondo que a sua apresentação ( $A1$ ) obteve a classificação de "Bom" (75), a elegância técnica da solução ( $B2$ ) obteve a classificação de "Excelente" (100) e o capricho ( $B3$ ) obteve a classificação de "Ruim" (25), a nota final é:

$$Nota_{trab} = 30. \left( \frac{75}{100} \right) + \frac{70}{100} \cdot \left( 60. \frac{75}{100} + 20. \frac{100}{100} + 20. \frac{25}{100} \right) = 67,5 \text{ pontos em 100 pontos}$$

Este mesmo estudante fez quase todas as atividades em sala avaliadas pelo tutor, obtendo  $N_{traDiarios} = 90$  pontos e fez também o requisito 11 adicional, obtendo 10 pontos adicionais. Portanto, a nota final deste estudante neste(s) sprint(s) é de:

$$Nota_{final} = 0,9.67,5 + 0,1.90 + 10 = 79,75 \Rightarrow 80 \text{ pontos}$$